

ТЕОРИЯ ГРАФОВ

Информационные деревья



Ассоциативная память

- В практическом программировании для организации хранения данных и доступа к ним часто используется механизм, который обычно называют **ассоциативной памятью**.
- При использовании ассоциативной памяти данные делятся на порции (называемые *записями*), и с каждой записью ассоциируется **ключ**.
- Ключ — это значение из некоторого линейно упорядоченного множества, а записи могут иметь произвольную природу и различные размеры.
- Доступ к данным осуществляется по *значению ключа*, которое обычно выбирается простым, компактным и удобным для работы.

Примеры организации ассоциативной памяти

- Толковый словарь или энциклопедия: записью является *словарная статья*, а ключом — *заголовок* словарной статьи (обычно его выделяют жирным шрифтом).
- **Адресная книга**: ключом является имя абонента, а записью — адресная информация (телефон(ы), почтовый адрес и т. д.).
- **Банковские счета**: ключом является номер счета, а записью — финансовая информация (которая может быть очень сложной).

Ассоциативная память должна поддерживать по меньшей мере три основные операции:

- 1) добавить (ключ, запись);
- 2) найти (ключ, запись);
- 3) удалить (ключ).

Эффективность ассоциативной памяти

- Эффективность каждой операции зависит от *структуры данных*, используемой для представления ассоциативной памяти.
- Эффективность ассоциативной памяти в целом зависит от **соотношения частоты выполнения различных операций** в данной конкретной программе.
- НЕВОЗМОЖНО указать способ организации ассоциативной памяти, который оказался бы наилучшим во всех возможных случаях.

Способы реализации ассоциативной памяти

Для представления ассоциативной памяти используются следующие основные структуры данных:

- *неупорядоченный массив;*
- *упорядоченный массив;*
- *дерево сортировки* — бинарное дерево, каждый узел которого содержит ключ (и указатель на запись) и обладает следующим свойством: значения ключа во всех узлах левого поддеревья меньше, а во всех узлах правого поддеревья — больше, чем значение ключа в узле;
- *таблица расстановки (или хэш-таблица).*

Неупорядоченные/упорядоченные массивы

При использовании неупорядоченного массива алгоритмы реализации операций ассоциативной памяти очевидны:

- 1. Операция «добавить (ключ, запись)» реализуется добавлением записи в конец массива.
- 2. Операция «найти (ключ, запись)» реализуется проверкой в цикле всех записей в массиве.
- 3. Операция «удалить (ключ)» реализуется поиском удаляемой записи, а затем перемещением последней записи на место удаляемой.

Для упорядоченного массива имеется эффективный алгоритм двоичного поиска. Реализация остальных операций очевидна.

Алгоритм бинарного (двоичного) поиска

Вход: упорядоченный массив A : **array** $[1..n]$ **of record** $k : key; i : info$ **end record**; ключ $a : key$.

Выход: индекс записи с искомым ключом a в массиве A или 0, если записи с таким ключом нет.

$b := 1$ { начальный индекс части массива для поиска }

$e := n$ { конечный индекс части массива для поиска }

while $b \leq e$ **do**

$c := (b + e) / 2$ { индекс проверяемого элемента (округленный до целого) }

if $A[c].k > a$ **then**

$e := c - 1$ { продолжаем поиск в первой половине }

else if $A[c].k < a$ **then**

$b := c + 1$ { продолжаем поиск во второй половине }

else

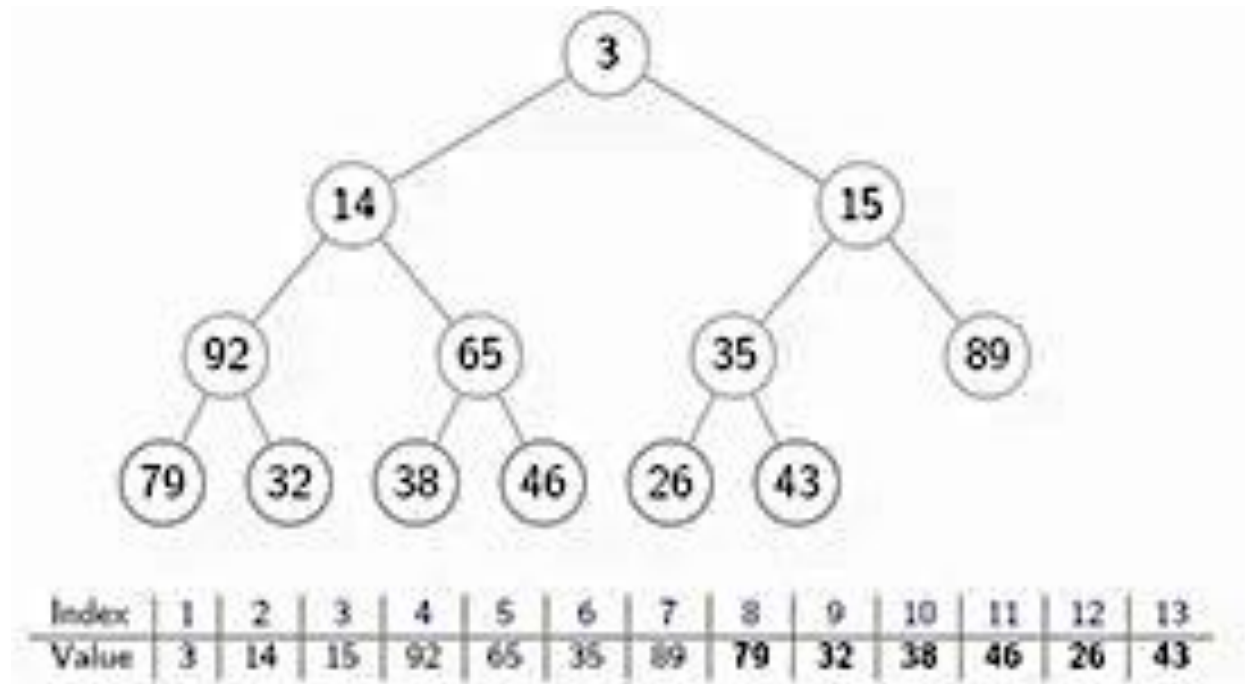
return c { нашли искомый ключ }

end if

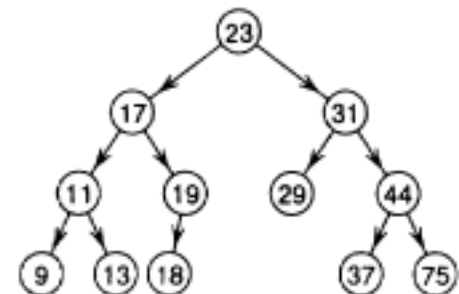
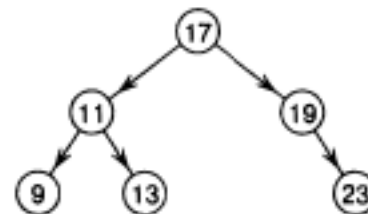
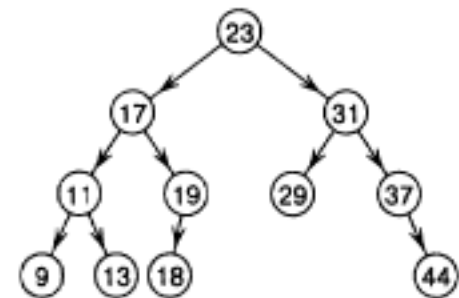
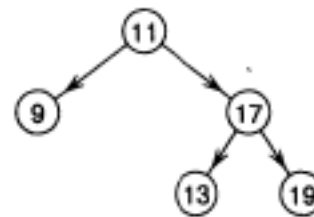
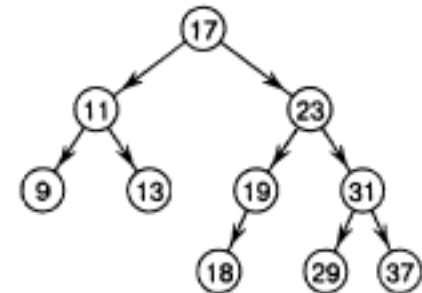
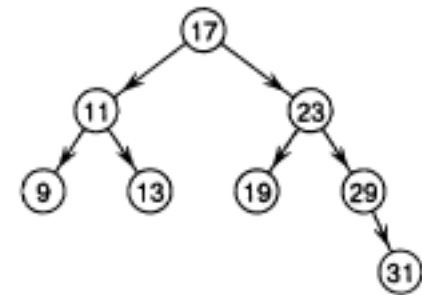
end while

return 0 { искомого ключа нет в массиве }

Дерево сортировки. Пример



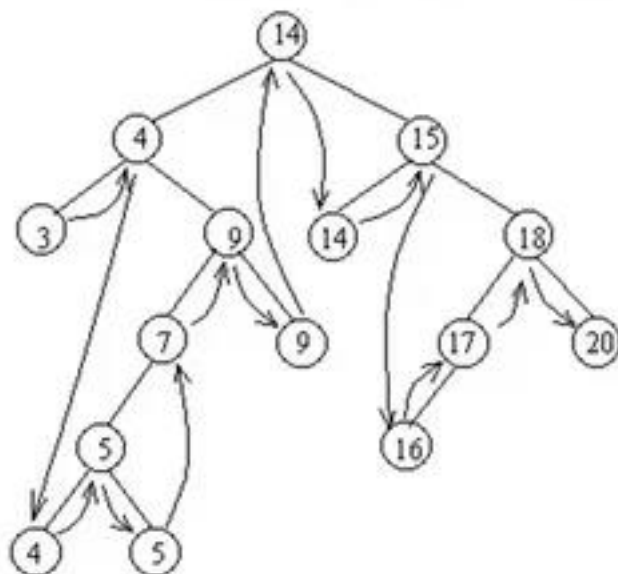
Сбалансированное дерево:
 9, 11, 13, 17, 19,
 23, 29, 31, 37, 18,
 44, 75.



Дерево сортировки. Пример

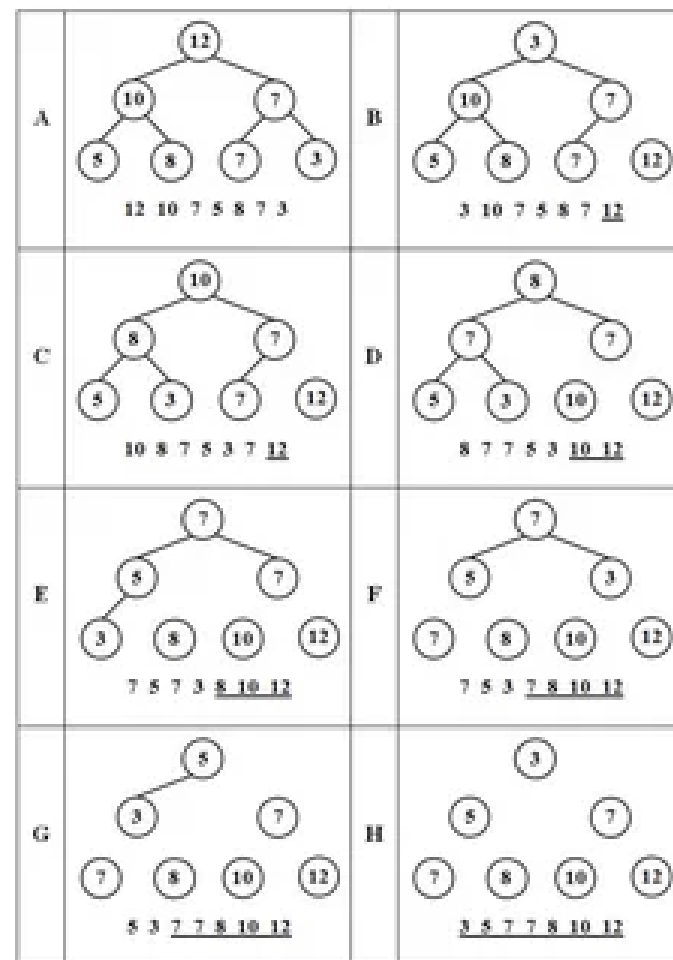
Исходный массив:

14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5



Прохождение в симметричном порядке:

3, 4, 4, 5, 5, 7, 9, 9, 14, 14, 15, 16, 17, 18, 20



Другой пример дерева сортировки

- «У моего компьютера есть чип на материнской плате»



Поиск в дереве сортировки

Вход: дерево сортировки T , заданное указателем на корень; ключ a : *key*.

Выход: указатель p на найденный узел или **nil**, если в дереве нет такого ключа.

$p := T$ { указатель на проверяемый узел }

while $p \neq \text{nil}$ **do**

if $a < p.i$ **then**

$p := p.l$ { продолжаем поиск слева }

else if $a > p.i$ **then**

$p := p.r$ { продолжаем поиск справа }

else

return p { нашли узел }

end if

end while

return **nil** { искомого ключа нет в дереве }

Вставка в дерево сортировки

Вход: дерево сортировки T , заданное указателем на корень; ключ a : *key*.

Выход: модифицированное дерево сортировки T .

```
if  $T = \text{nil}$  then
     $T := \text{NewNode}(a)$  { первый узел в дереве }
    return  $T$ 
end if
 $p := T$  { указатель на текущий узел }
while true do
    if  $a = p.i$  then
        return  $T$  { в дереве уже есть такой ключ }
    end if
    if  $a < p.i$  then
        if  $p.l = \text{nil}$  then
             $p.l := \text{NewNode}(a)$  { создаём новый узел }
            return  $T$  { и подцепляем его к  $p$  слева }
        else
             $p := p.l$  { продолжаем поиск места для вставки слева }
        end if
    else
        if  $a > p.i$  then
            if  $p.r = \text{nil}$  then
                 $p.r := \text{NewNode}(a)$  { создаём новый узел }
                return  $T$  { и подцепляем его к  $p$  справа }
            else
                 $p := p.r$  { продолжаем поиск места для вставки справа }
            end if
        end if
    end if
end while
```

Удаление из дерева сортировки

Вход: дерево сортировки T , заданное указателем на корень; ключ a : *key*.

Выход: модифицированное дерево сортировки T .

Find(T, a, p, q, s) { поиск удаляемого узла }

if $p = \text{nil}$ then

 return T { нет такого узла — ничего делать не нужно }

end if

if $p.r = \text{nil}$ then

 Delete($p, q, p.l, s$) { случай 1, рис. 9.12, *слева* }

else

$u := p.r$

 if $u.l = \text{nil}$ then

$u.l := p.l$

 Delete(p, q, u, s) { случай 2, рис. 9.12, *в центре* }

 else

$w := u; v := u.l$

 while $v.l \neq \text{nil}$ do

$w := v; v := v.l$

 end while

$p.i := v.i$

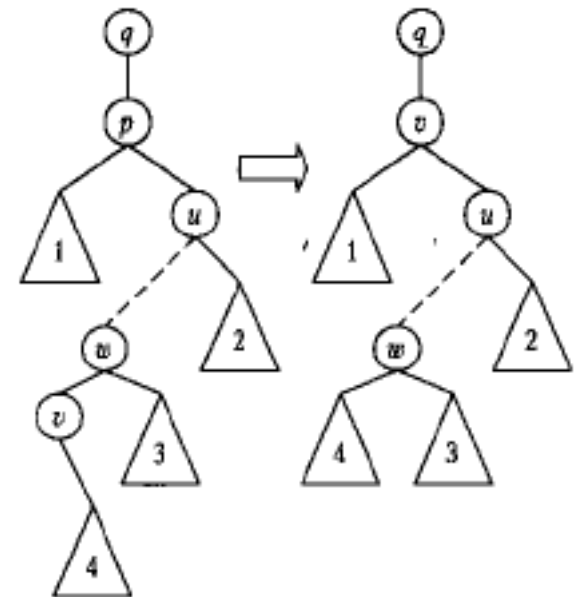
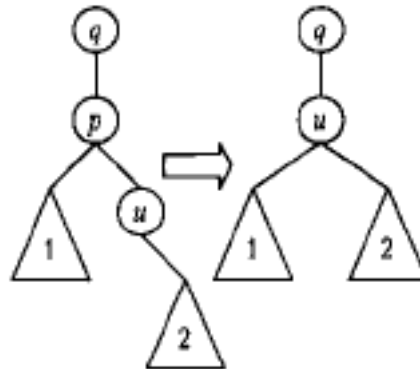
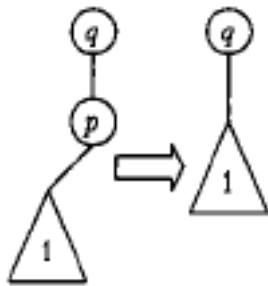
 Delete($v, w, v.r, -1$) { случай 3, рис. 9.12, *справа* }

 end if

end if

return T

Удаление узла (три случая)



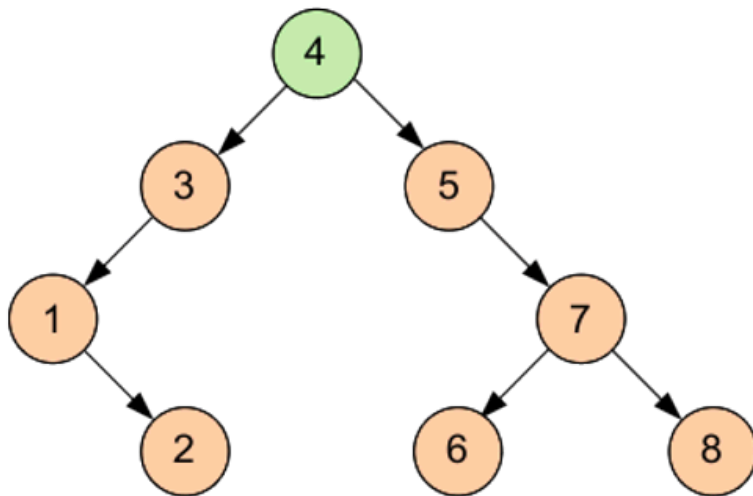
Сравнение представлений ассоциативной памяти

- Эффективность операций с деревом сортировки ограничена сверху высотой дерева.
- Дерево сортировки может расти неравномерно. Например, если при загрузке дерева исходные данные уже упорядочены, то полученное дерево будет право- или левوليнейным и окажется даже менее эффективным, чем неупорядоченный массив.

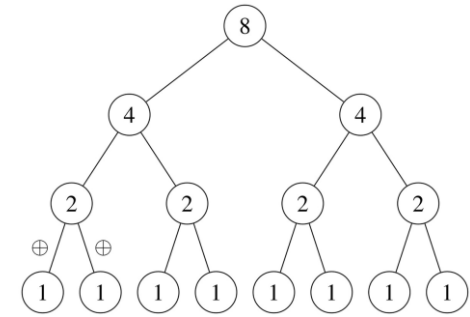
	Неупорядоченный массив	Упорядоченный массив	Дерево сортировки
Добавить	$O(1)$	$O(n)$	$O(\log_2(n))..O(n)$
Найти	$O(n)$	$O(\log_2(n))$	$O(\log_2(n))..O(n)$
Удалить	$O(n)$	$O(n)$	$O(\log_2(n))..O(n)$

Выровненные и заполненные деревья

- Бинарное дерево называется выровненным, если все листья находятся на одном (последнем) уровне. В выровненном дереве все ветви имеют одну длину, равную высоте дерева, однако выровненное дерево не всегда является эффективным деревом сортировки.
- Бинарное дерево называется заполненным, если все узлы, степень которых меньше 2, располагаются на одном или двух последних уровнях. Другими словами, в заполненном дереве T все ярусы, кроме, может быть, последнего, заполнены.
- Заполненное дерево имеет наименьшую возможную для данного p высоту h .



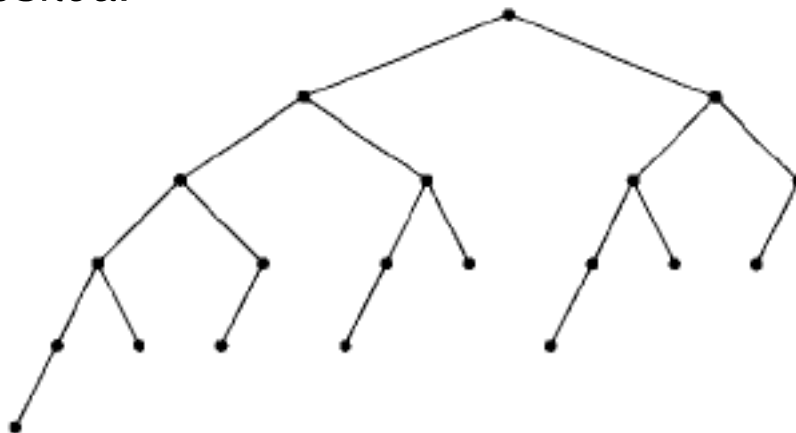
Полное дерево



- Выровненное заполненное бинарное дерево называется полным. В полном бинарном дереве все листья находятся на последнем уровне и все узлы, кроме листьев, имеют полустепень исхода 2.
- В полном дереве все ярусы заполнены.
- Полное бинарное дерево высотой h имеет $2^{h+1} - 1$ узлов.
- Заполненные деревья дают наибольший возможный эффект при поиске. Однако известно, что вставка/удаление в заполненное дерево может потребовать полной перестройки всего дерева, и, таким образом, трудоёмкость операции в худшем случае составит $O(p)$.

Сбалансированные деревья

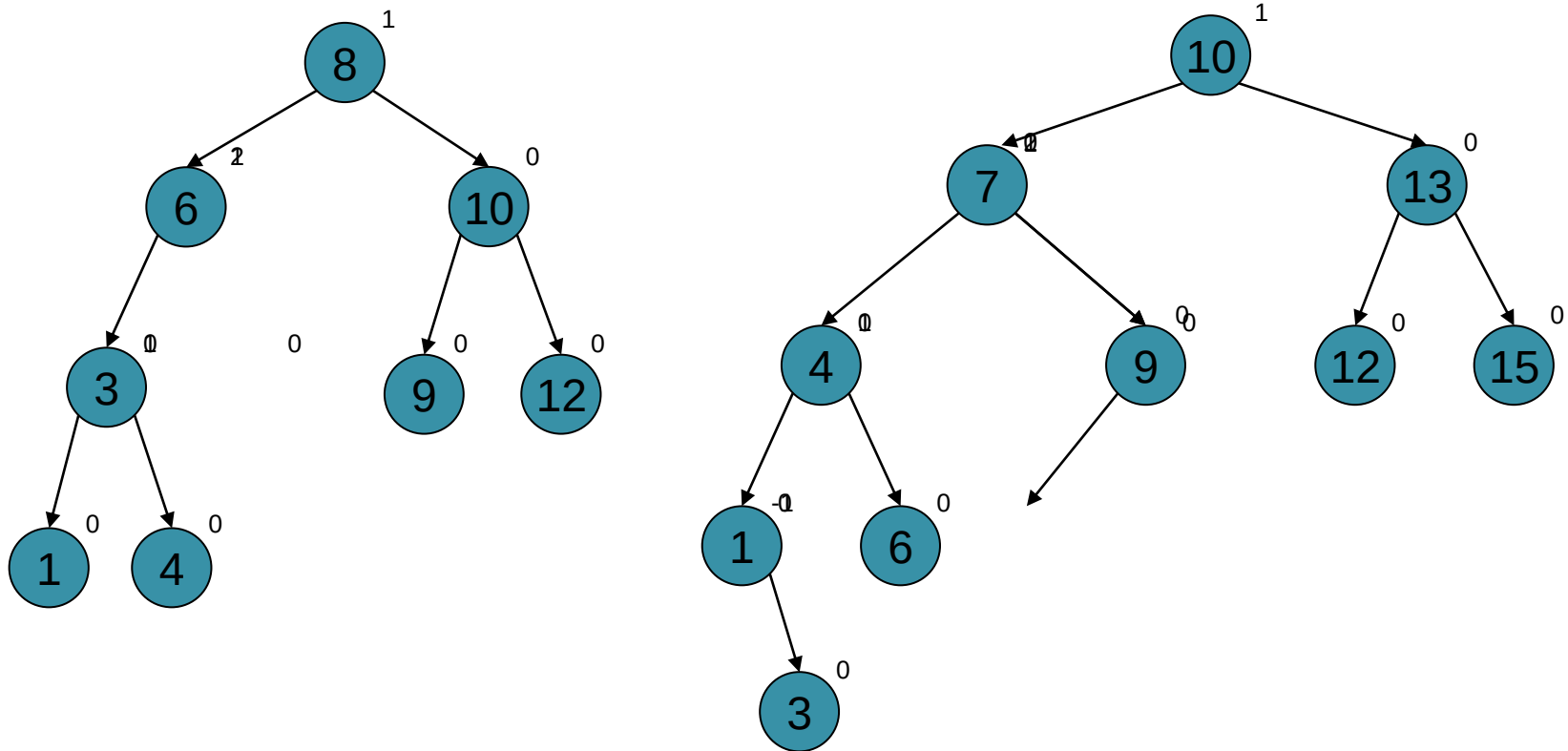
- (Бинарное) дерево называется *подровненным деревом*, или *АВЛ-деревом* (Адельсон-Вельский и Лайдис, 1962), или **сбалансированным по высоте** деревом, если для любого узла высоты левого и правого поддеревьев различаются не более чем на единицу.
- Кроме сбалансированных по высоте, рассматривают и другие классы деревьев, например, *сбалансированные по весу* деревья, в которых для каждого узла количества узлов в левом и правом поддеревьях находятся в определённом соотношении.
- Для сбалансированного по высоте бинарного дерева $h < 2 \log_2 p$.
- *Самое первое сбалансированное двоичное дерево с самобалансировкой.*



Сбалансированные деревья

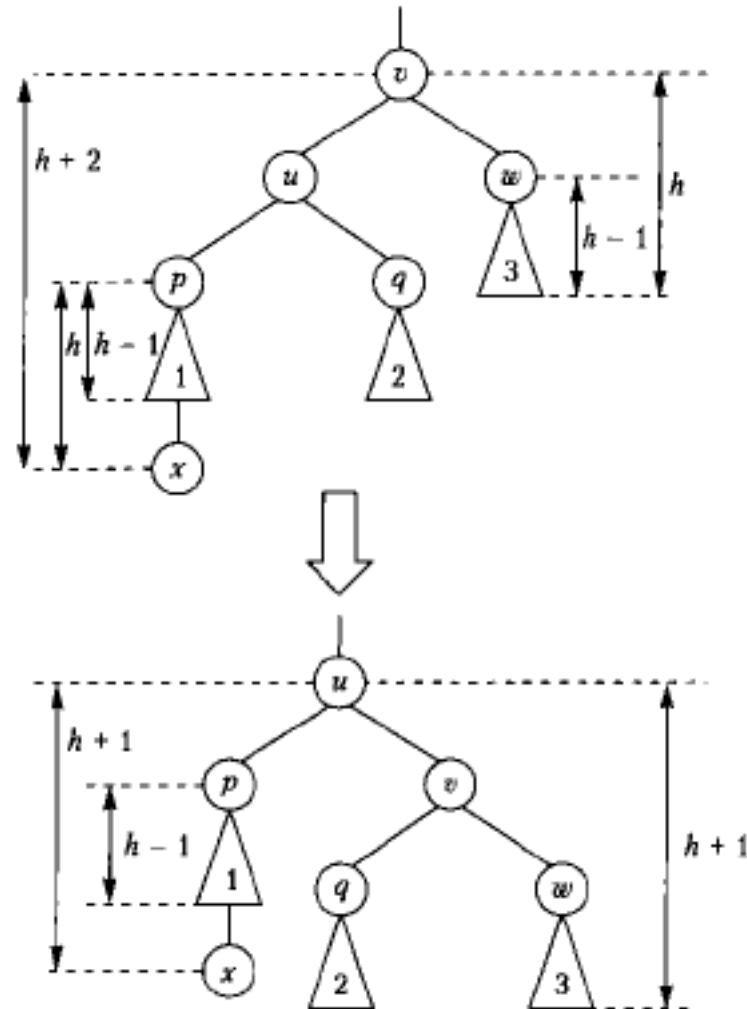
- *Сбалансированные деревья* уступают *заполненным деревьям* по скорости поиска (менее чем в два раза), однако их преимущество состоит в том, что известны алгоритмы вставки узлов в сбалансированное дерево и удаления из него, которые сохраняют сбалансированность и в то же время при перестройке дерева затрагивают только конечное число узлов.
- Поэтому во многих случаях сбалансированное дерево оказывается *наилучшим вариантом* представления дерева сортировки.

Сбалансированные по высоте деревья

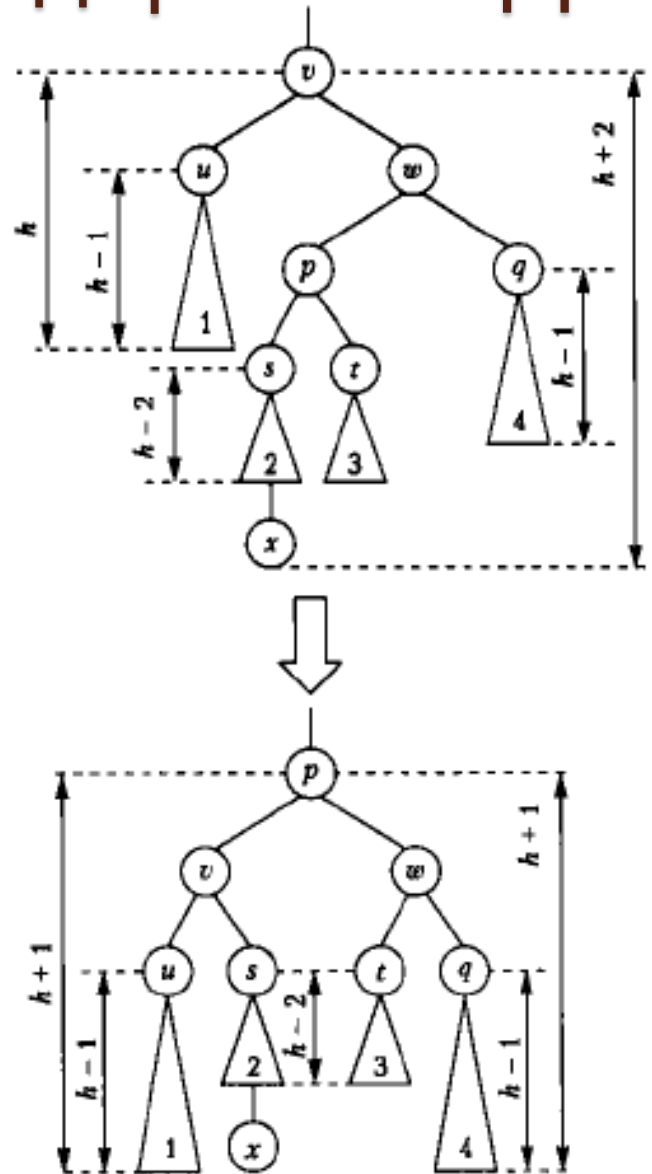


В общем случае трудно придумать алгоритм, позволяющий сбалансировать произвольное дерево, но можно предложить алгоритм балансировки дерева после добавления или удаления одного узла.

Балансировка деревьев. Простое вращение



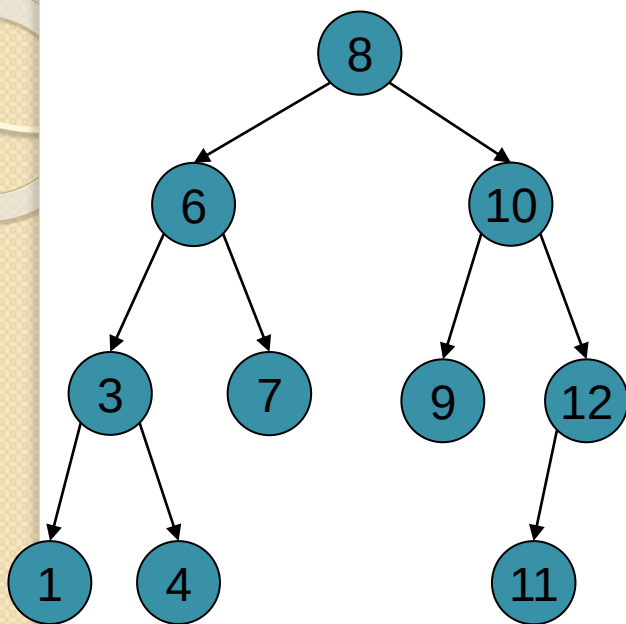
Балансировка деревьев. Двойное вращение



Сбалансированные деревья. Итог

- Накладывает на дерево следующее ограничение: у любой вершины высоты левого и правого поддеревьев **должны отличаться не более** чем на 1.
- Имеет сложность $O(\log_2 n)$ на все основные операции. Для работы необходимо хранить в каждой вершине разницу между высотами левого и правого поддеревьев. Так как она не превосходит 1, достаточно использовать **2 бита** на вершину.

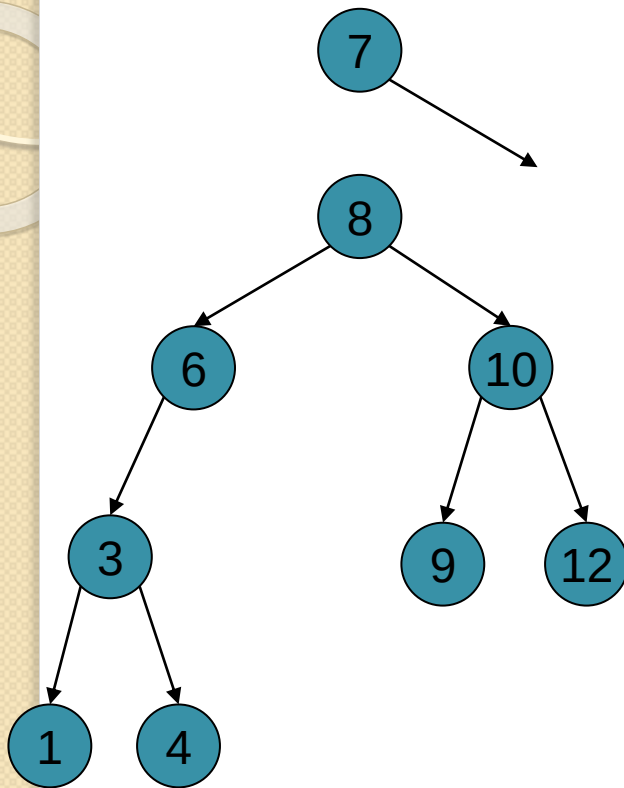
Добавление данных в лист дерева



```
public void addLeaf(Comparable key, Object obj) {
    TreeNode newNode = new TreeNode(key, obj);
    TreeNode current = root;
    TreeNode pred = null;

    while (current != null) {
        int res = key.compareTo(current.key);
        if (res == 0) return;
        else {
            pred = current;
            current =
                (res < 0 ?
                 current.left :
                 current.right);
        }
    }
    if (pred == null)
        root = newNode;
    else {
        if (key.compareTo(pred.key) < 0)
            pred.left = newNode;
        else
            pred.right = newNode;
    }
}
```

Добавление данных в корень дерева

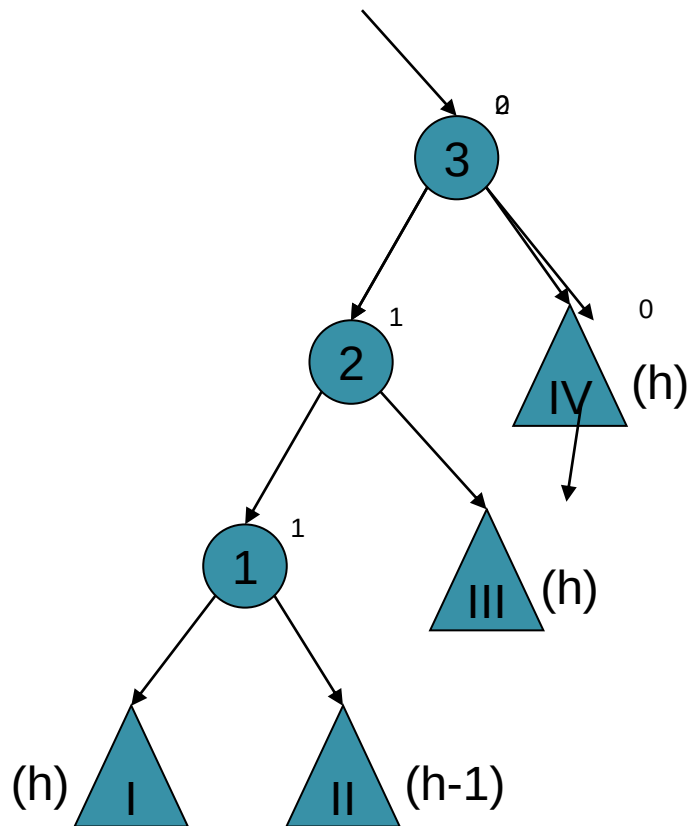


```
public void addRoot(Comparable key, Object obj) {  
    TreeNode newNode = new TreeNode(key, obj);  
    addRoot(key, root, newNode, 0, newNode, 1);  
    root = newNode;  
}
```

```
private void addRoot(Comparable key,  
    TreeNode current,  
    TreeNode leftRef,  
    int lrLeft,  
    TreeNode rightRef,  
    int lrRight) {  
    if (current == null) {  
        if (lrLeft == 0)  
            leftRef.left = null;  
        else  
            leftRef.right = null;  
        if (lrRight == 0)  
            rightRef.left = null;  
        else  
            rightRef.right = null;  
    } else if (key.compareTo(current.key) > 0) {  
        if (lrLeft == 0)  
            leftRef.left = current;  
        else  
            leftRef.right = current;  
        addRoot(key, current.right,  
            current, 1, rightRef, lrRight);  
    } else {  
        if (lrRight == 0)  
            rightRef.left = current;  
        else  
            rightRef.right = current;  
        addRoot(key, current.left,  
            leftRef, lrLeft, current, 0);  
    }  
}
```

Алгоритм «простого поворота»

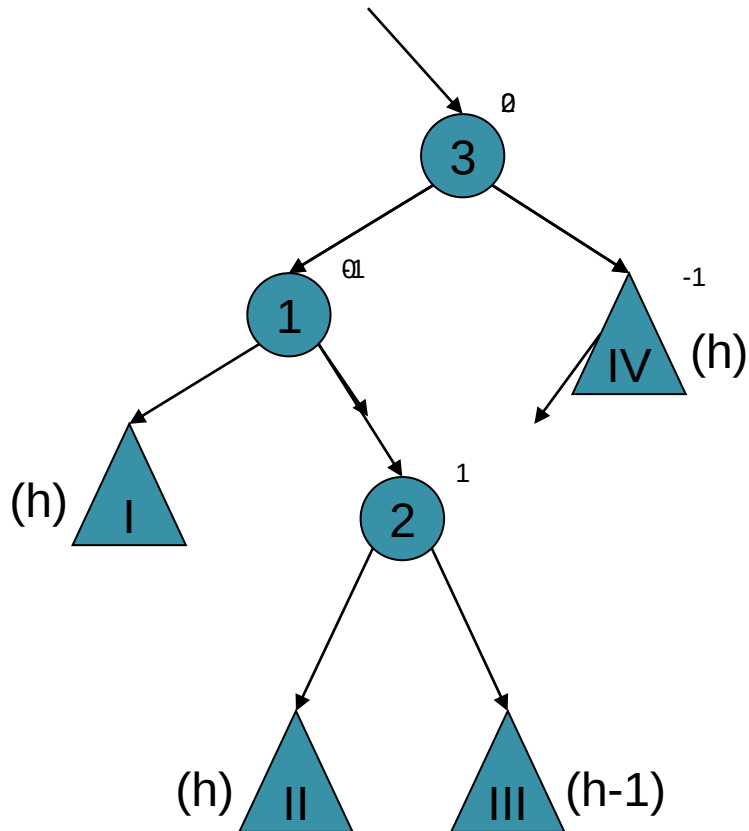
1. Отсоединение поддеревьев
2. Поворот
3. Присоединение поддеревьев



$(I) < (1) < (II) < (2) < (III) < (3) < (IV)$

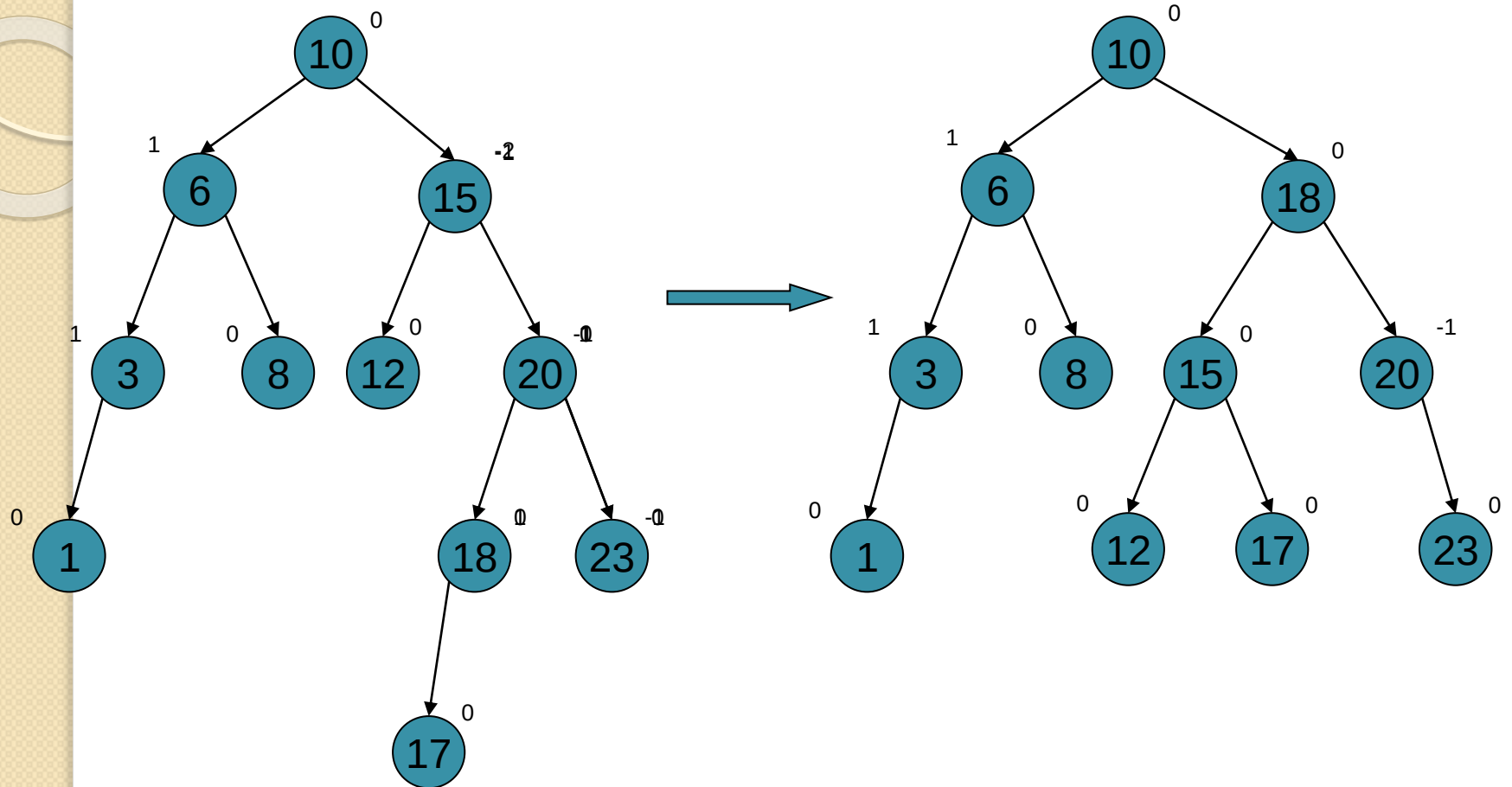
Алгоритм «двойного поворота»

1. Отсоединение поддеревьев
2. Поворот
3. Присоединение поддеревьев



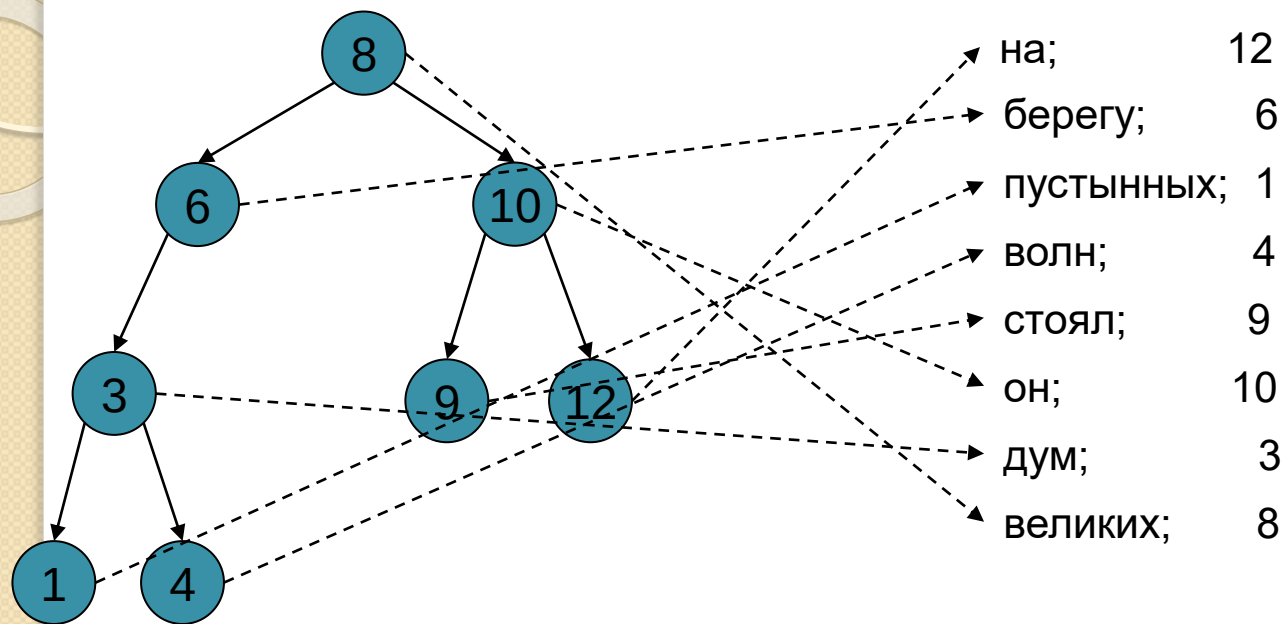
$(I) < (1) < (II) < (2) < (III) < (3) < (IV)$

Пример балансировки



Dictionary operation	singly unsorted	double unsorted	singly sorted	doubly sorted
Search(L, k)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert(L, x)	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Delete(L, x)	$O(n)^*$	$O(1)$	$O(n)^*$	$O(1)$
Successor(L, x)	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Predecessor(L, x)	$O(n)$	$O(n)$	$O(n)^*$	$O(1)$
Minimum(L)	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Maximum(L)	$O(n)$	$O(n)$	$O(1)^*$	$O(1)$

Индексация данных



С помощью поиска по индексу можно получить ответы на вопросы:

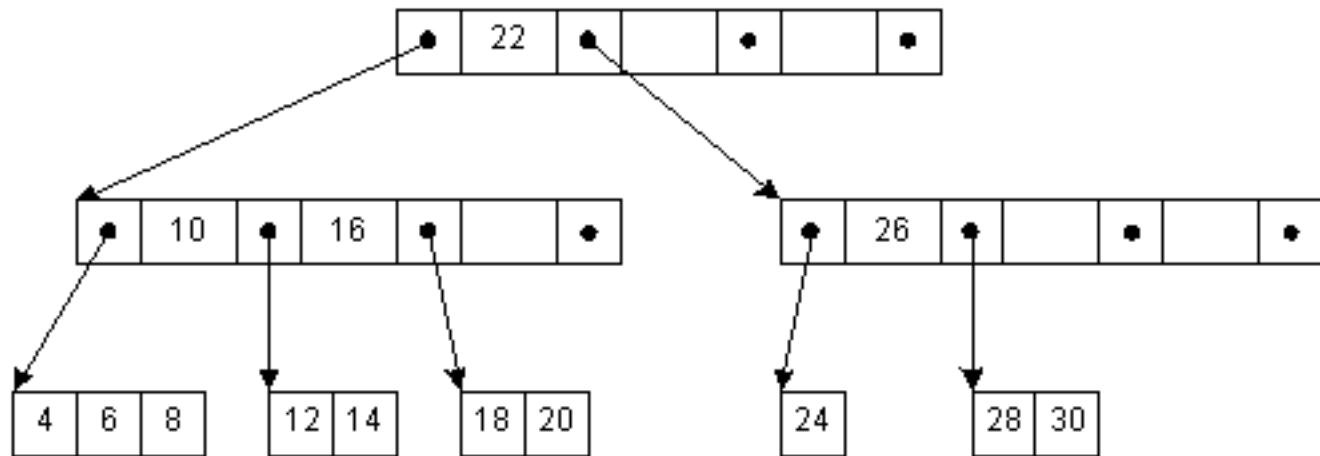
- Какое слово встречается ровно 6 раз?
- Какие слова встречаются больше 10 раз?
- Какое слово встречается чаще всего?

В-дерево

- Сбалансированное, сильно ветвистое дерево во внешней памяти.
 - *Сбалансированность* означает, что длина любых двух путей от корня до листьев совпадает.
 - *Ветвистость* дерева — свойство каждого узла дерева ссылаться на большое число узлов-потомков.
- Применяется для уменьшения количества операций обмена: уравнивается размер узла с размером сектора и сгруппировываются вместе несколько ключей в каждом узле.
- Данное самобалансирующееся двоичное дерево впервые описано Рудольфом Байеров в 1972 году.

В-дерево

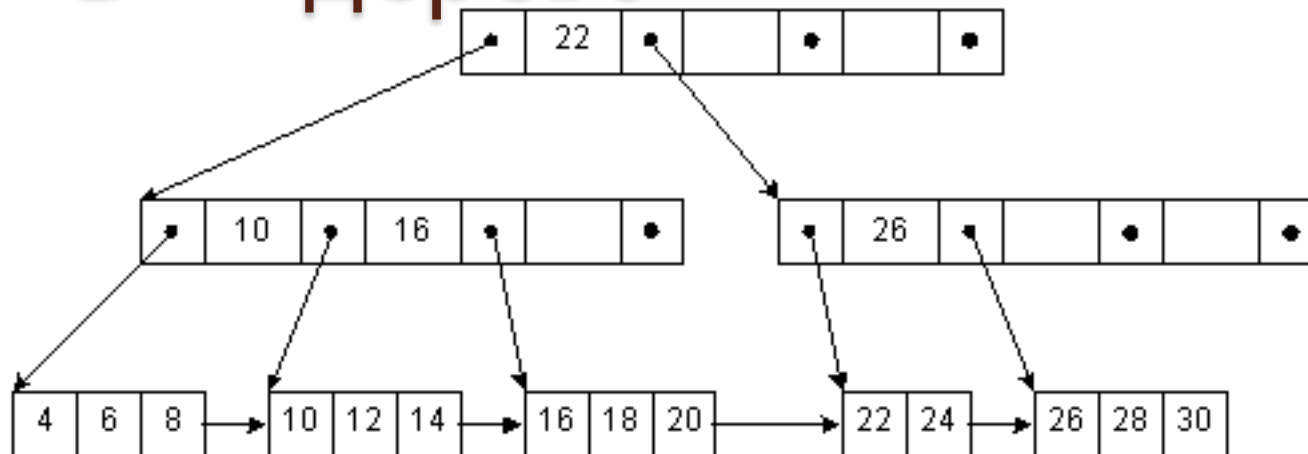
- Структура В-дерева применяется для организации индексов во многих современных СУБД.
- В-дерево может применяться для структурирования (индексирования) информации на жёстком диске (как правило, метаданных).



В-дерево

- **В-дерево** – данные хранятся в любом узле.
 - Ключи и данные хранятся как во внутренних узлах, так и в листьях (концевых узлах).
- **В*-дерево** – данные хранятся в любом узле.
 - Единственное отличие – узлы заполняются на $2/3$. Это приводит к лучшему использованию места, занимаемого деревом, и чуть лучшей производительности.
- **В⁺-дерево** – данные хранятся только в листьях.
 - Все ключи хранятся в листьях, там же хранится и информационная часть узла.
 - Во внутренних узлах хранятся *копии ключей* – они помогают искать нужный лист.
 - У указателей смысл немножко не такой, как при работе с обычными В-деревьями: левый указатель ведет к ключам, которые *меньше* заданного значения, правый - ключам, которые *больше или равны*.
- *Они стали более известны как **красно-черные** деревья после некоторой новой упрощенной формулировки, предложенной Леонидасом Гимбасом и Робертом Седжвиком в 1978 году.*

B+ - дерево



- Произвольный доступ к записи реализуется посредством малого количества подопераций (обращения к физическим блокам).
- В среднем достаточно эффективно реализуются операции включения и удаления записей; при этом сохраняется естественный порядок ключей с целью последовательной обработки, а также соответствующий баланс дерева для обеспечения быстрой произвольной выборки.
- Неизменная упорядоченность по ключу обеспечивает возможность эффективной пакетной обработки.
- Основной недостаток B-деревьев состоит в отсутствии для них эффективных средств выборки данных (т.е. метода обхода дерева), упорядоченных по отличному от выбранного ключа.

Пример

за_горани_за_лесами_за_широкими_морями_не_на_небе_на_земле_жил_
старик_в_одном_селе_у_крестьянина_три_сына_старший_умный_был_
детина_средний_сын_и_так_и_сяк_младший_вовсе_был_дурак_братья_
сеяли_пшеницу_да_возили_в_град_столицу_знать_столица_та_была_
недалече_от_села_там_пшеницу_продавали_деньги_счетом_принимали_
и_с_набитой_суной_возвращались_домой_

Пример

```
L1  горами_жила_за_земле_лесами_морями_на_не_небе_широкими_-----
L1  горами_жила_за_земле_лесами_-----
L2  морями_на_не_небе_старик_широкими_-----
B1  L1:лесами_+L2:широкими_

L1  в_горами_жила_за_земле_крестьянина_лесами_-----
L2  морями_на_не_небе_одном_селе_старик_-----
L3  старший_сына_три_у_широкими_-----
B1  L1:лесами_+L2:старик_+L3:широкими_

L1  был_в_вовсе_горами_детина_дурак_-----
L4  жила_за_земле_и_крестьянина_лесами_-----
L2  младший_морями_на_не_небе_одном_селе_средний_старик_-----
L3  старший_сын_сына_сяк_так_три_у_умный_широкими_-----
B1  L1:дурак_+L4:лесами_
B2  L2:старик_+L3:широкими_
B3  B1:лесами_+B2:широкими_
```

Пример

```
L1 братья_был_была_в_вовсе_возвращались_возили_-----
L7 горами_град_да_деньги_детина_домой_дурак_-----
L4 жил_за_земле_знать_и_крестьянина_лесами_-----
L2 младший_морями_на_набитую_не_небе_недалече_одном_-----
L5 от_принимали_продавали_пшеницу_-----
L8 с_села_селе_сеяли_средний_старик_-----
L3 старший_столица_столицу_сумой_счетом_сын_сына_-----
L6 сак_та_так_там_три_у_умный_широкими_-----
V1 L1:возили_+L7:дурак_+L4:лесами_
V2 L2:одном_+L5:пшеницу_+L8:старик_
V4 L3:сына_+L6:широкими_
V3 V1:лесами_+V2:старик_+V4:широкими_
```

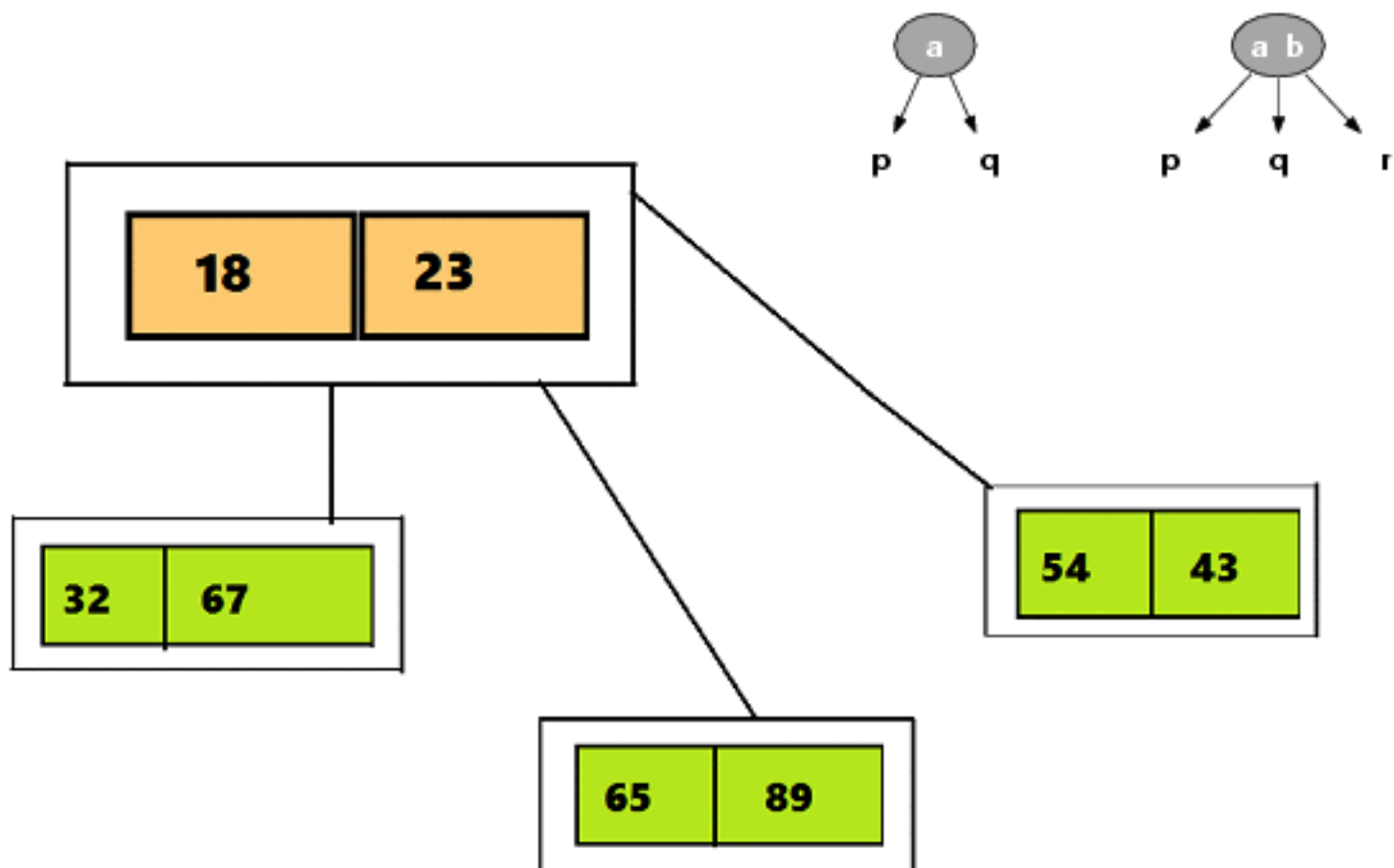
2-3-дерево

Структура данных, являющаяся В-деревом степени 1, которое содержит 2-вершины (вершины с одним полем и 2 потомками) или 3-вершины (вершины с 2 полями и 3 потомками).

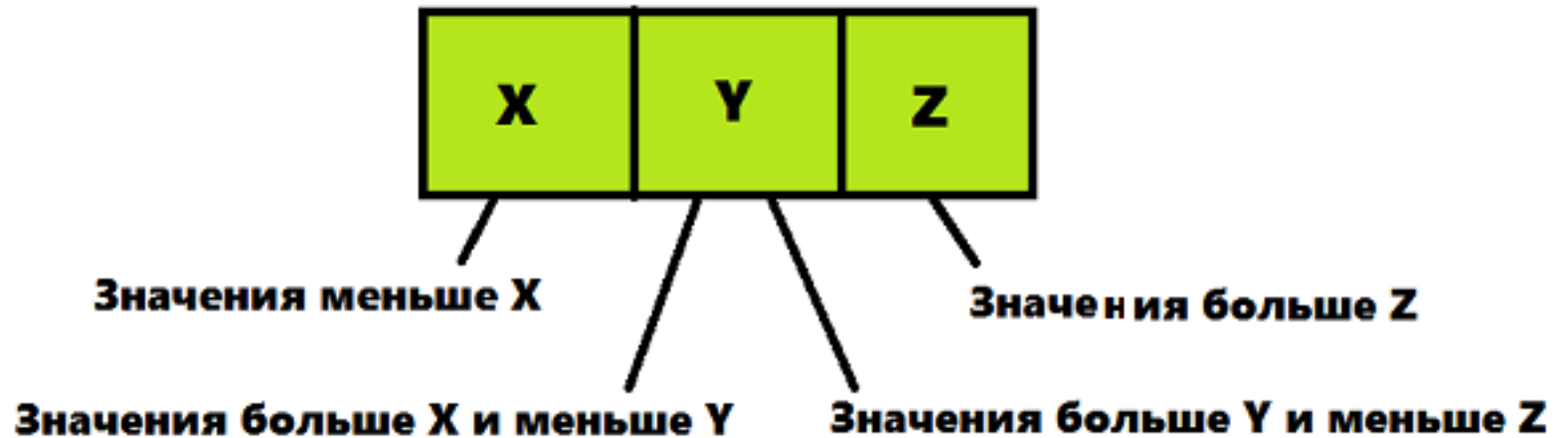
2-3 деревья сбалансированы: левое, правое и центральное поддеревья имеют одну и ту же высоту, и, таким образом, содержат равные (или почти равные) объемы данных.

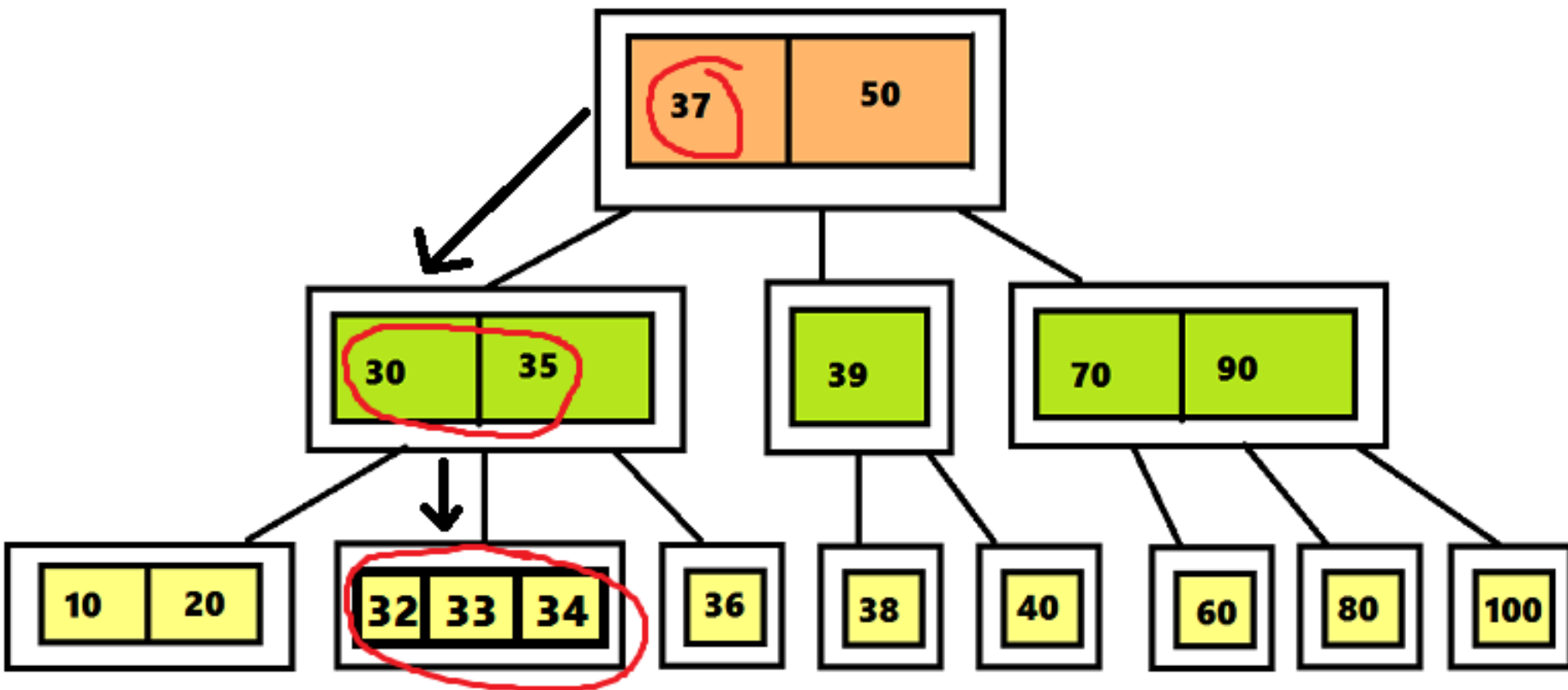
Свойства:

- Все нелистовые вершины содержат одно поле и 2 поддерева или 2 поля и 3 поддерева.
- Все листовые вершины находятся на одном уровне (на нижнем уровне) и содержат 1 или 2 поля.
- Все данные отсортированы (по принципу двоичного дерева поиска).



2-3-4 дерево





Красно-черное дерево

Другие названия: Red-black tree, RB-tree.

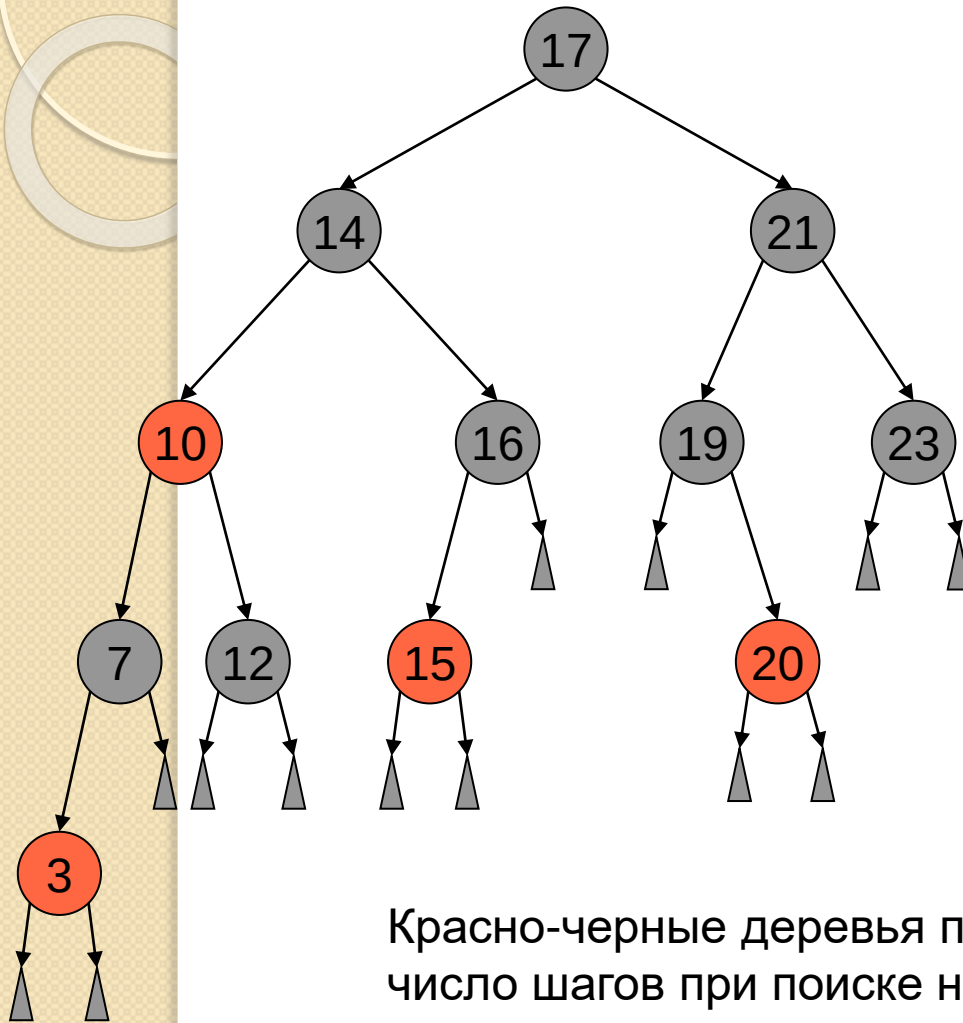
Бинарное дерево, баланс которого достигается за счет поддержания раскраски вершин в *два цвета*, подчиняющейся следующим правилам:

- Каждый узел покрашен либо в **черный**, либо в **красный** цвет.
- Листьями объявляются **NIL**-узлами (т.е. "виртуальные" узлы, наследники узлов, которые обычно называют листьями; на них "указывают" NULL указатели). Листья покрашены в **черный** цвет.
- Каждый **красный** узел имеет **черного** родителя (если узел красный, то оба его потомка черны).
- На всех ветвях дерева, ведущих от его корня к листьям, число **черных** узлов *одинаково*.

Для реализации этого вида сбалансированных деревьев нужно в каждой вершине хранить дополнительно 1 бит информации (цвет).

Все основные операции с красно-черным деревом можно реализовать за $O(h)$, то есть $O(\log_2 n)$.

Красно-черное дерево



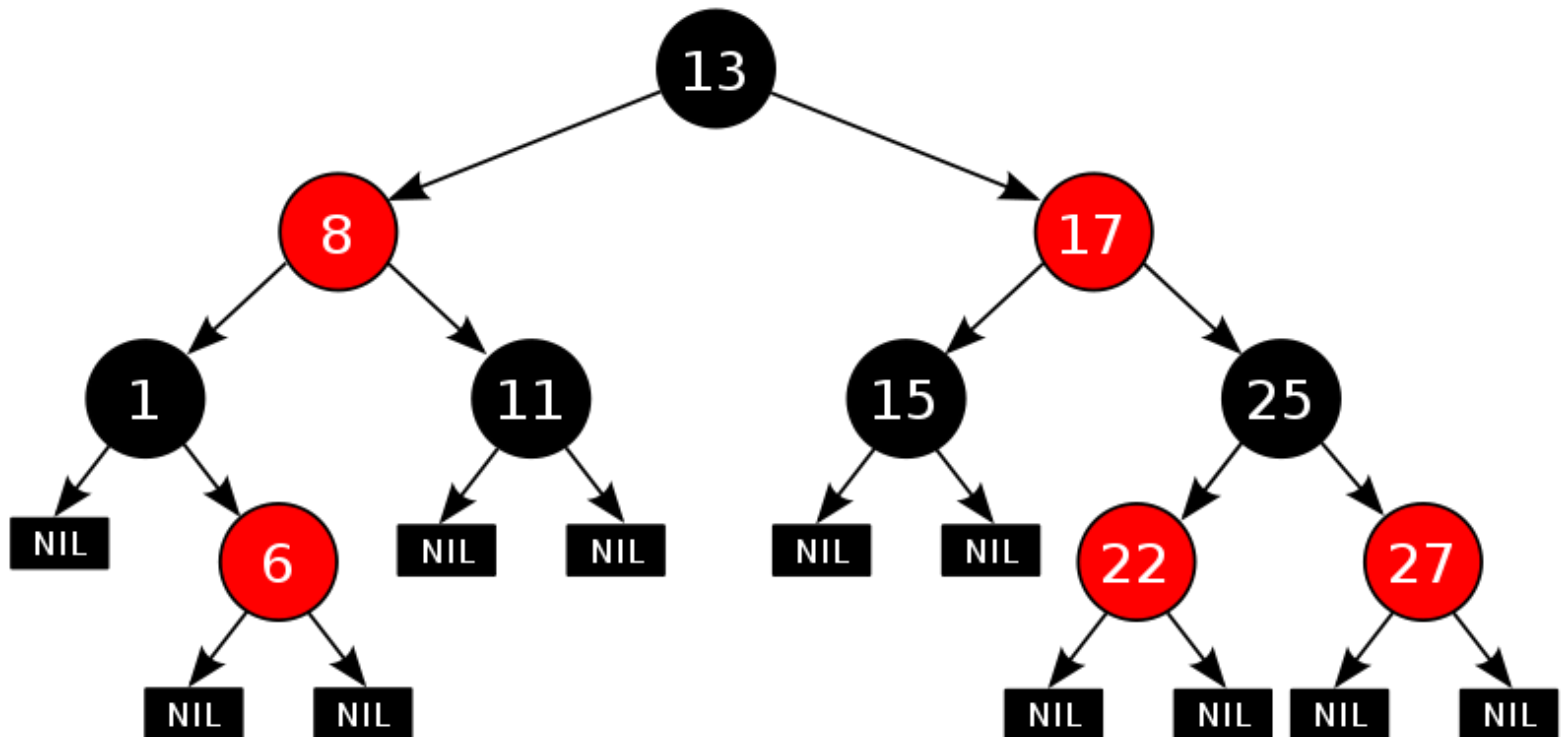
Свойства красно-черных деревьев

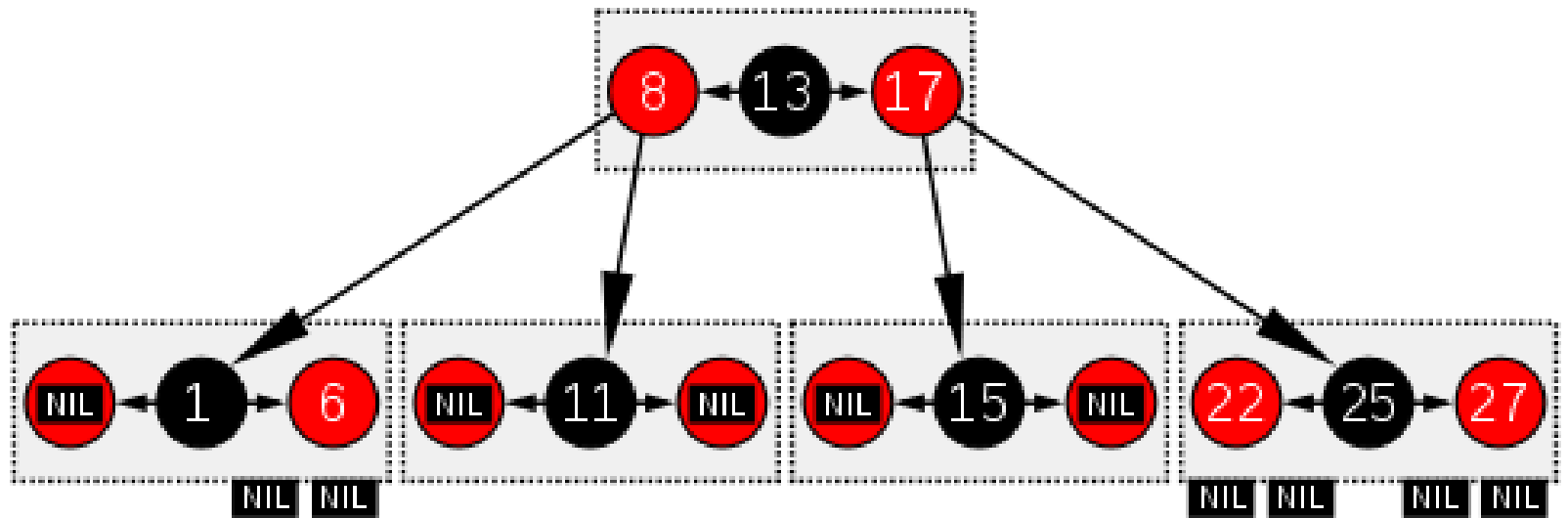
- Корень и пустые узлы всегда имеют черный цвет
- Красная вершина не может иметь красных потомков
- Максимальные «черные» длины путей, ведущих из корня к листьям (пустым узлам), одинаковы

Красно-черные деревья поиска достаточно эффективны: число шагов при поиске не превышает $2 \log_2 n$

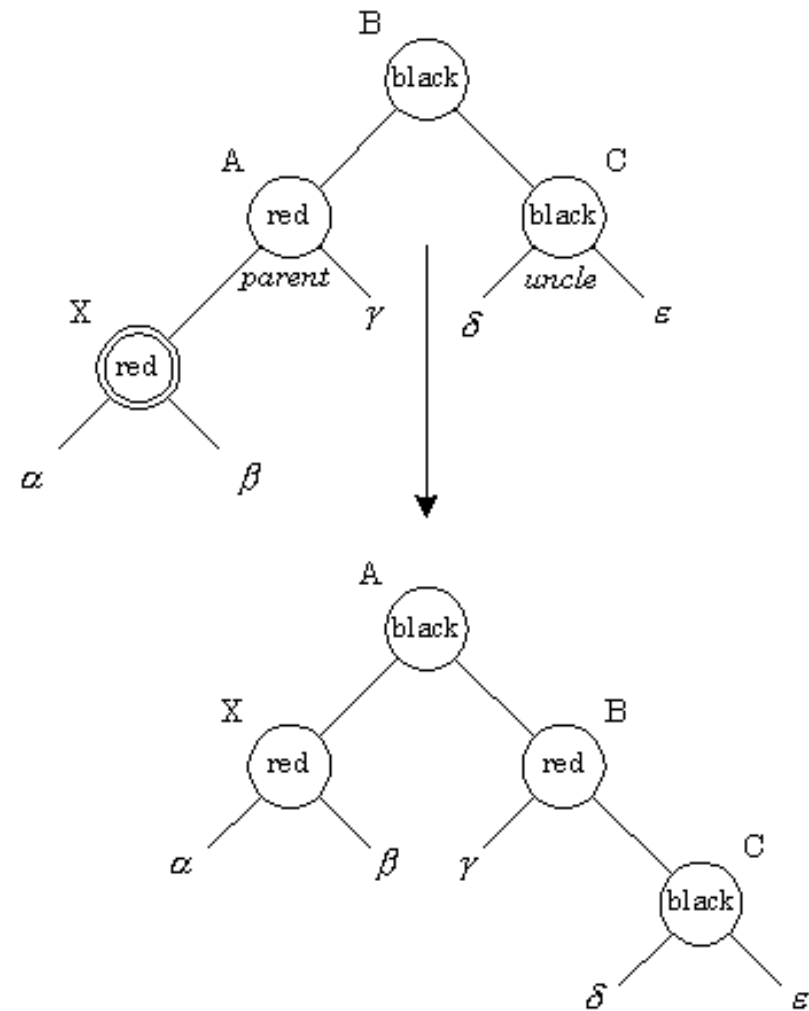
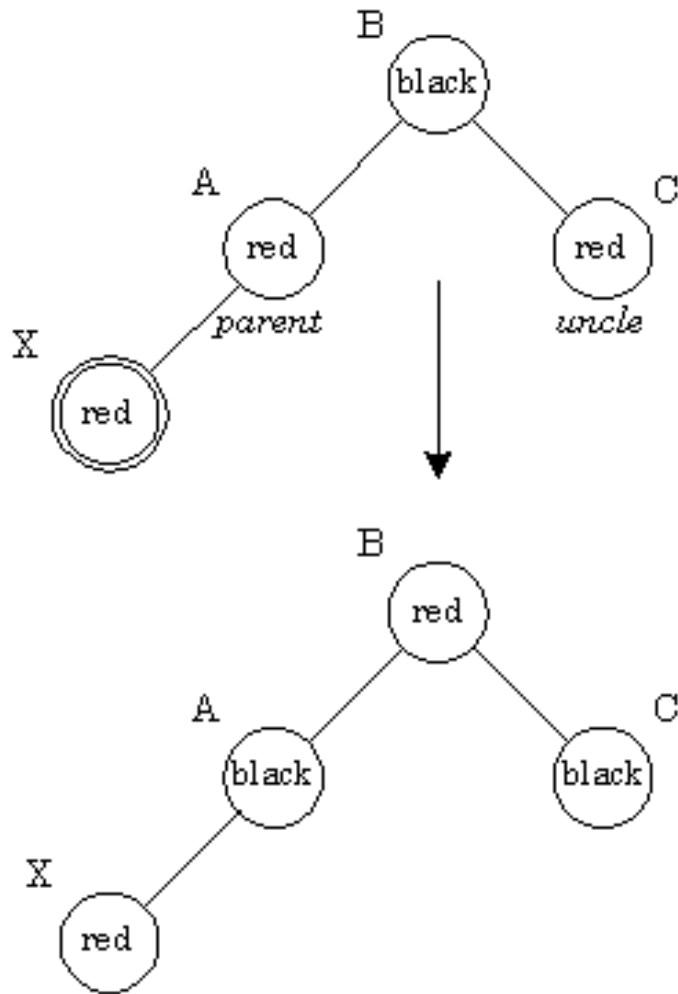
Существуют эффективные процедуры добавления и удаления узлов, которые не нарушают свойств красно-черных деревьев

Красно-черное дерево

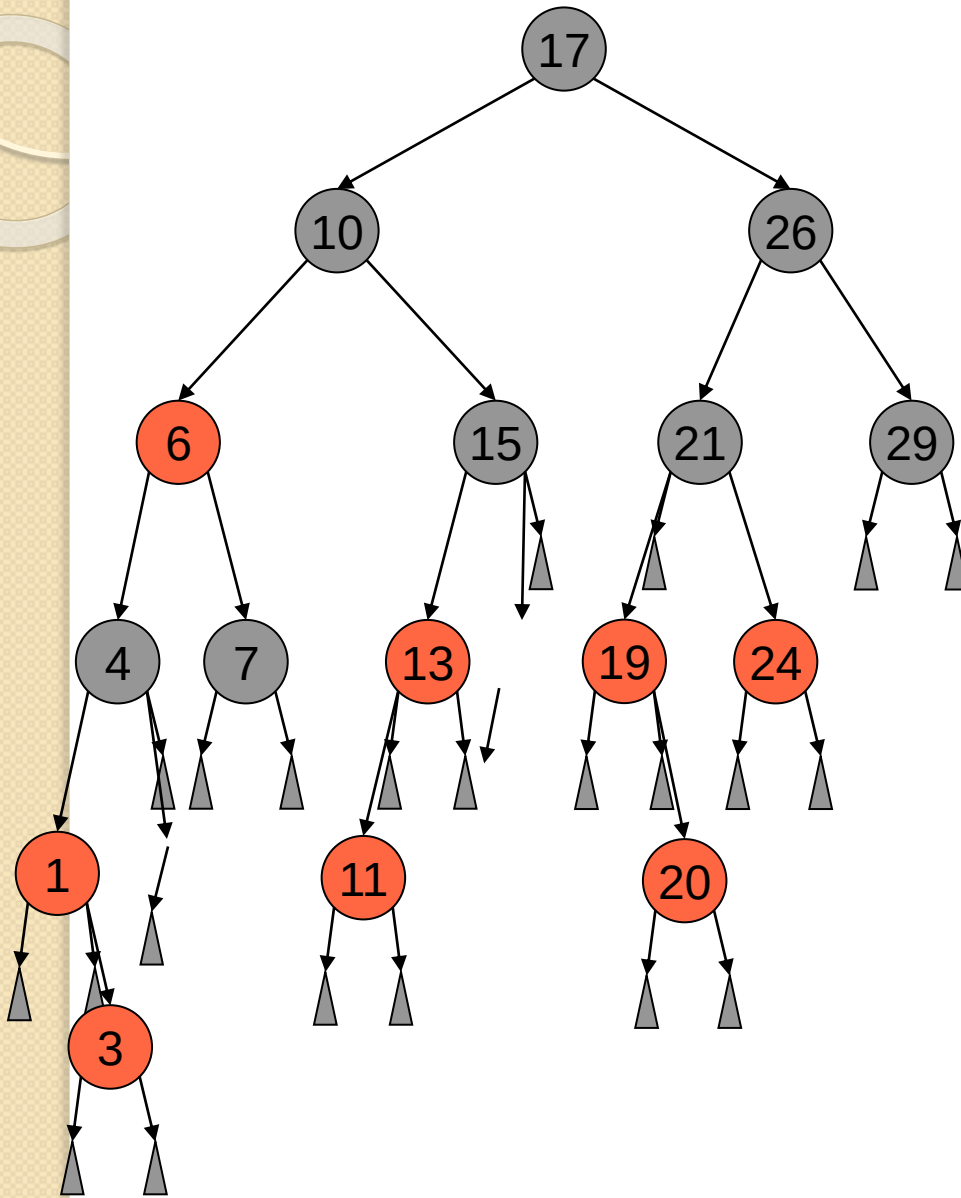




Вставка в красно-черное дерево



Алгоритм добавления ключа в красно-черное дерево



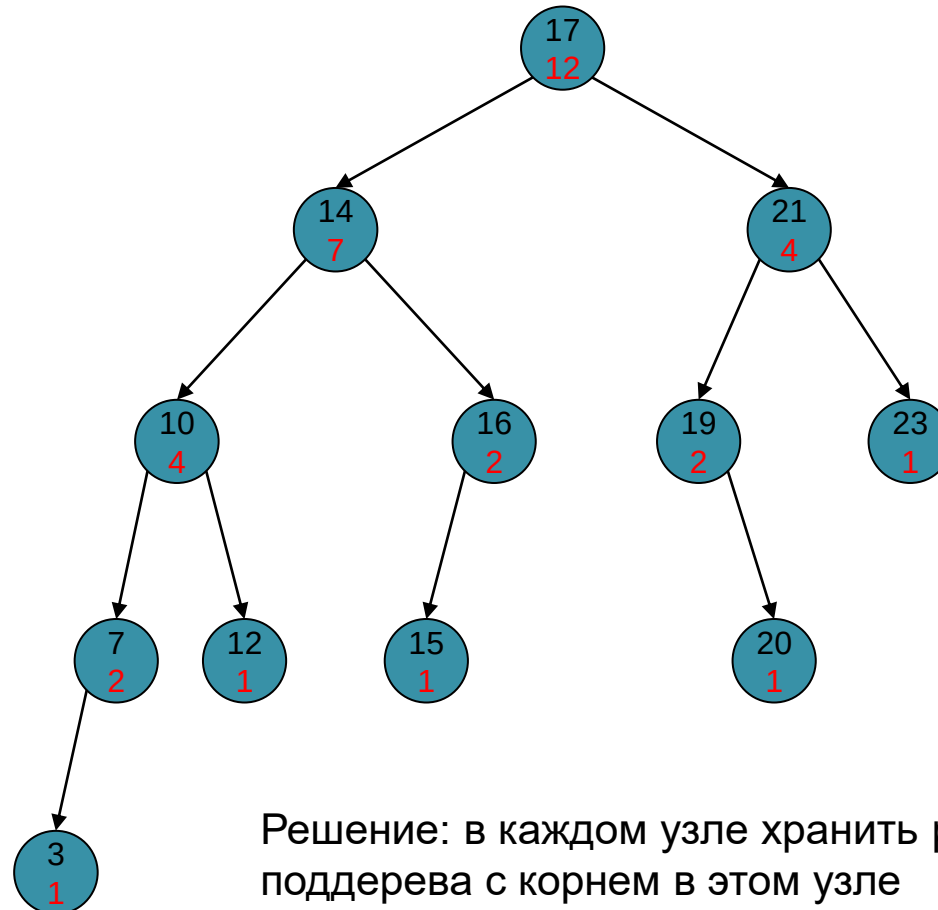
1. Добавляем ключ 19
2. Добавляем ключ 20
3. Добавляем ключ 11
4. Добавляем ключ 3...

Хранение дополнительной информации в деревьях

Задача: по заданному порядковому номеру найти узел дерева; и наоборот, по заданному узлу найти его порядковый номер.

Массив:

0	3
1	7
2	10
3	12
4	14
5	15
6	16
7	17
8	19
9	20
10	21
11	23



Решение: в каждом узле хранить размер поддеревя с корнем в этом узле

Алгоритмы работы с индексами в двоичных деревьях с размерами

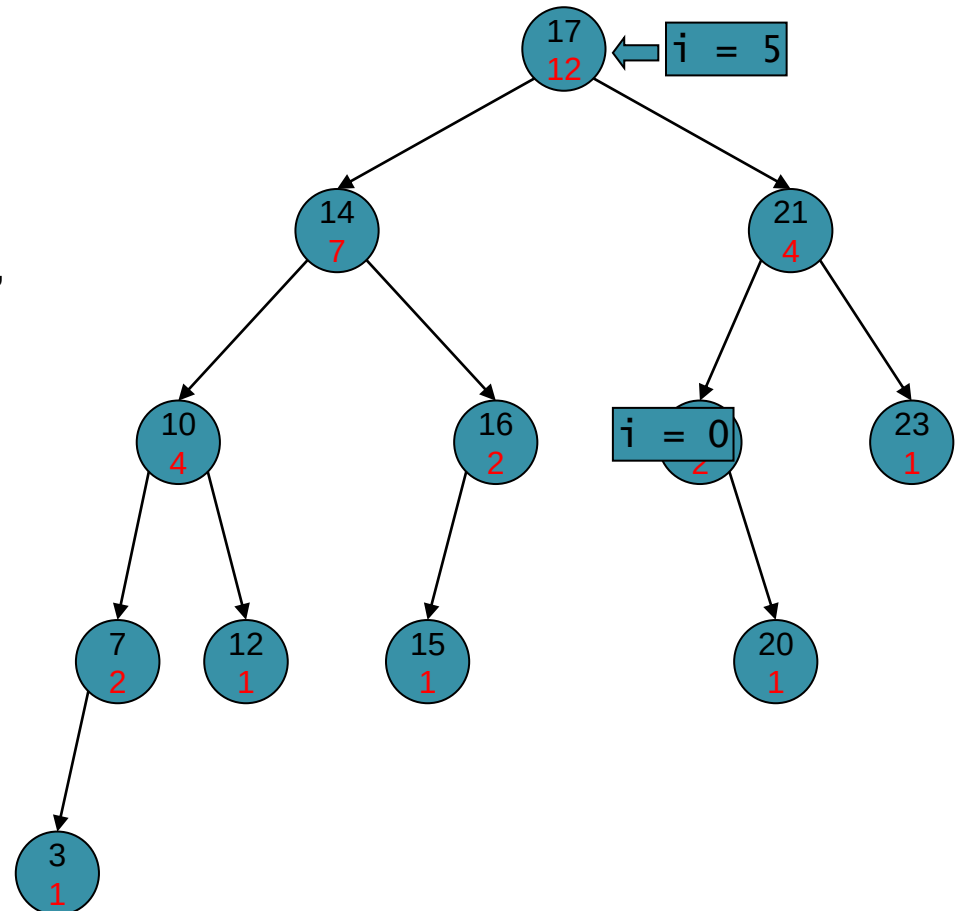
Поиск позиции по индексу:

`TreeNode findByIndex(int i)`

1. Начинаем цикл поиска с корня
`current = root;`
2. Если в левом поддереве ($== i$) узлов,
`return current;`
3. Если в левом поддереве ($> i$) узлов,
`current = current.left;`
4. Если в левом поддереве ($< i$) узлов,
`i -= (current.left.size + 1);`
`current = current.right;`

Вход: $i == 5$

Выход: `key == 15`



Программа поиска позиции узла в дереве по индексу

```
class Tree {  
    private TreeNode root = null;  
  
    public static class TreeNode {  
        Comparable info;  
        int size;  
        TreeNode parent, left, right;  
    }  
}
```

```
private TreeNode findByIndex(int i) {  
    // Pre: root != null &&  
    //       0 <= i < root.size  
    TreeNode current = root;  
    for (;;) {  
        // Inv: 0 <= i < current.size  
        int sizeL = 0;  
        if (current.left != null)  
            sizeL = current.left.size;  
        if (i == sizeL)  
            return current;  
        else if (i < sizeL)  
            current = current.left;  
        else {  
            i -= sizeL+1;  
            current = current.right;  
        }  
    }  
}
```

Алгоритмы работы с индексами в двоичных деревьях с размерами

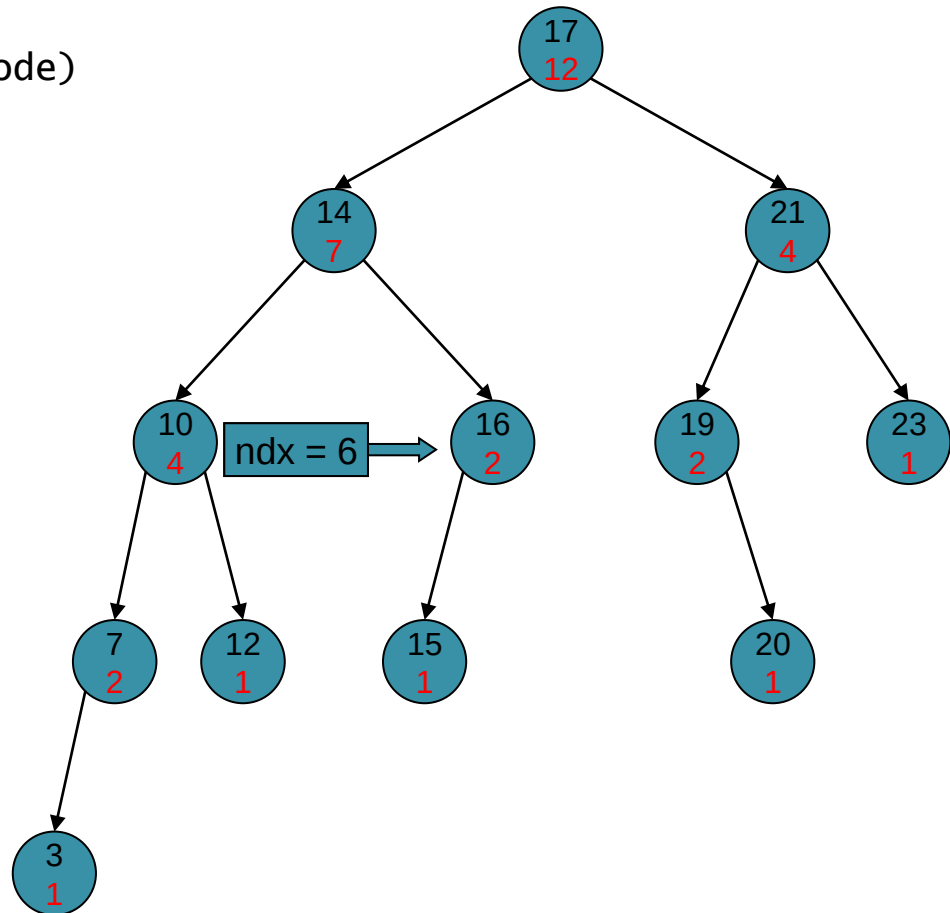
Поиск индекса по позиции:

`TreeNode findByPosition(TreeNode node)`

1. Начинаем цикл поиска с узла `node`;
`ndx = node.left.size`;
2. Если узел справа от родительского,
`ndx += (brother.size+1)`;
3. Переходим к родительскому узлу,
`node = node.parent`;

Вход: `key == 16`

Выход: `ndx == 6`



АА-дерево

Известны как красно-черные деревья с уклоном влево (left-leaning red-black trees).

Были предложены Арне Андерссоном в 1993 году и немного упрощены (и поименованы) Марком Алленом Вайсом в 2000 году.

Переформулированы (с другим алгоритмом изменения баланса Робертом Седжвиком в 2006 году.

Бинарное дерево, баланс которого достигается за счет поддержания раскраски вершин в *два цвета*, подчиняющейся следующим правилам:

- Модификация красно-черного дерева, в которой накладывается дополнительное ограничение: **красная** вершина может быть *только правым сыном*. Если красно-черное дерево изоморфно 2-3-4 дереву, то АА-дерево изоморфно 2-3 дереву.
- Из-за дополнительного ограничения операции реализуются проще, чем у красно-черного дерева (за счет *уменьшения* количества разбираемых случаев). Оценка на высоту деревьев остается прежней - $2 * \log_2 n$.



**СПАСИБО
ЗА ВНИМАНИЕ!**